

Introduction to System software

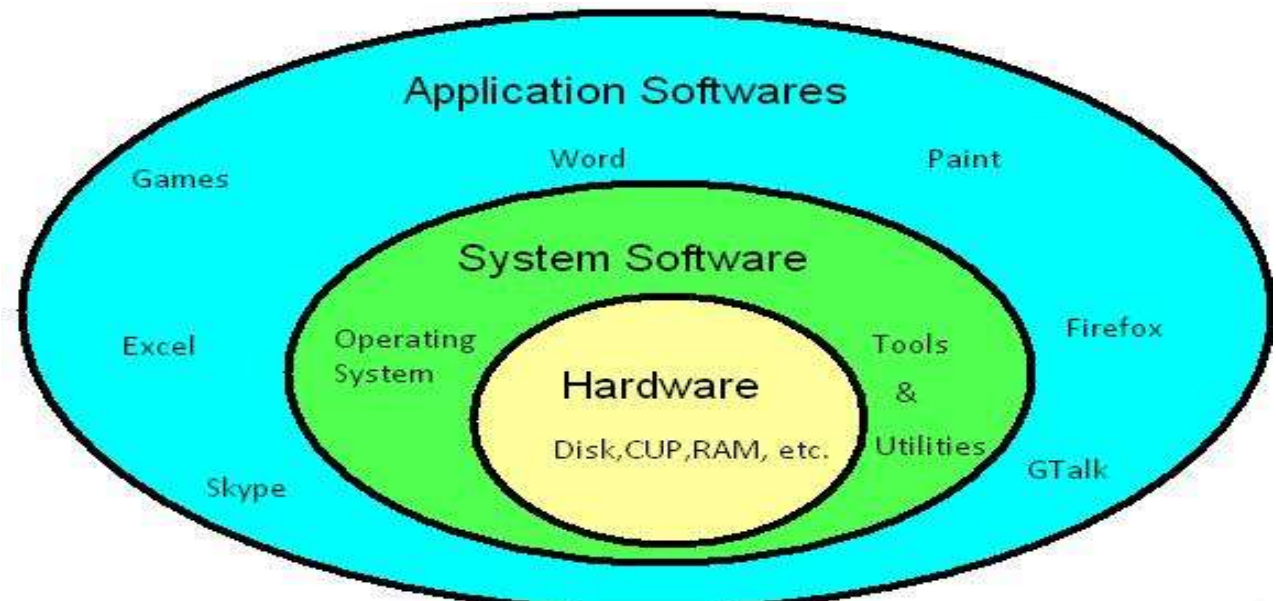
Module 1

System programming

- **System Programming** can be defined as act of building Systems Software using System Programming Languages.

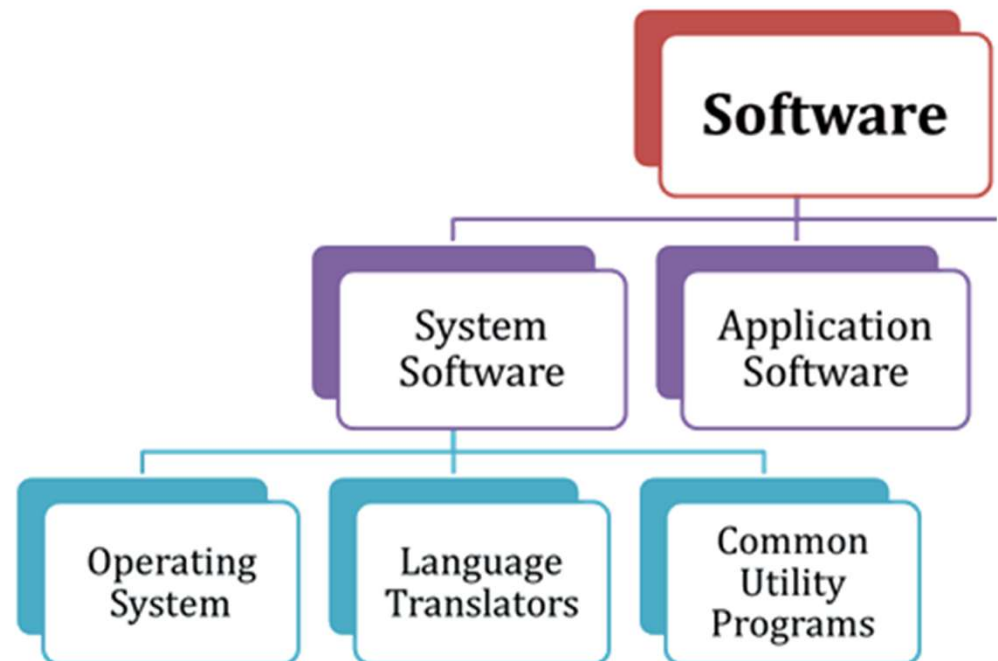
System programming

- According to Computer Hierarchy, one which comes at last is Hardware. Then it is System Programs, and finally Application Programs.
- System software serves as the **interface between the hardware and the end users.**



System programming

- There are two main types of software: systems software and application software.
 - Systems software includes the programs that are dedicated to managing the computer itself.
 - System Software is a set of programs that control and manage the operations of computer hardware. It also helps application programs to execute correctly.



System Programs

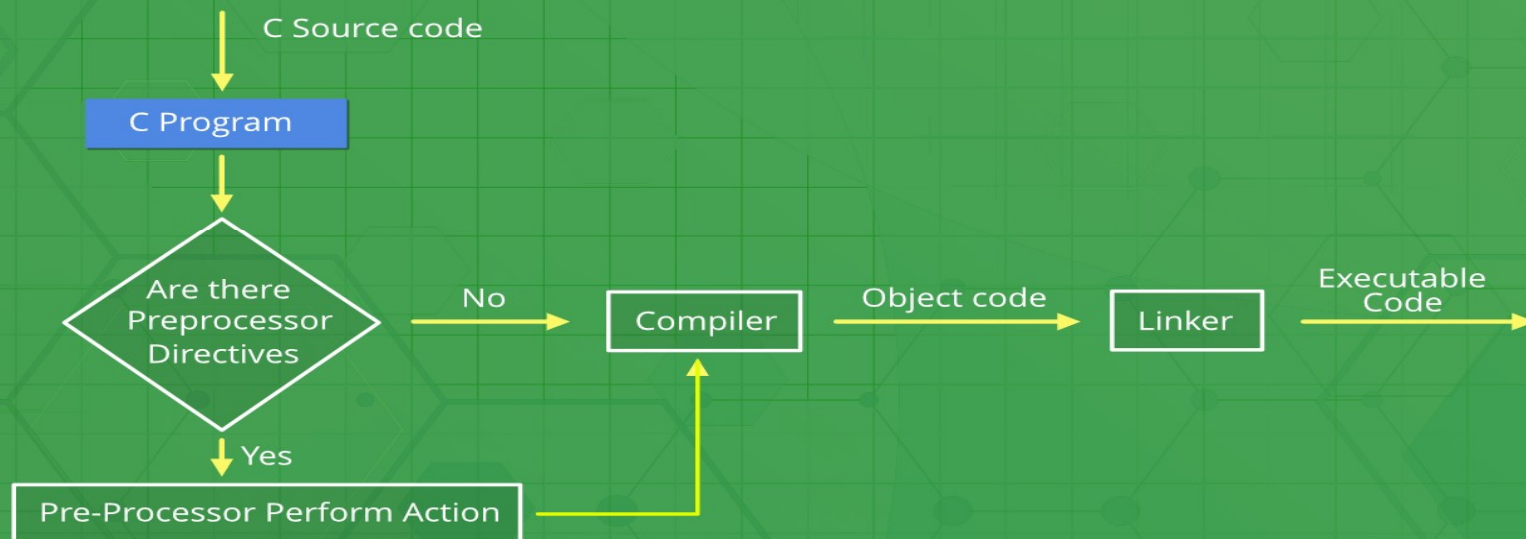
Various System Programs are :

- Assemblers
- Loaders
- Linkers
- Macro processor
- Compiler
- Interpreter
- Operating System
- Device drivers

Macro Processors or Pre Processors

- As the name suggests Preprocessors are programs that process our source code before compilation.

Preprocessor in C



Macro Processors or Pre Processors

There are 4 main types of pre-processor directives:

- Macros
- File Inclusion
- Conditional Compilation
- Other directives

Macros

- Macros are a piece of code in a program which is given some name.
 - Whenever this name is encountered by the compiler
 - The compiler replaces the name with the actual piece of code.
 - I.E. It replaces the name with the definition of the macro
- The '#define' directive is used to define a macro.
- Macro definitions need not be terminated by semi-colon(;;).

Macros

C program to illustrate macros

```
#include <stdio.h>
```

```
// Macro definition
```

```
#define LIMIT 5
```

```
int main()
```

```
{
```

```
    // Print the value of macro defined
```

```
    printf("The value of LIMIT is %d", LIMIT);
```

```
    return 0;
```

```
}
```

Output-

The value of LIMIT is 5

File Inclusion

- This type of preprocessor directive tells the compiler to include a file in the source code program.
- There are two types of files which can be included by the user in the program:
 - **Header File or Standard files**
 - **User defined files**

File Inclusion

Header File or Standard files:

- These files contains definition of pre-defined functions like `printf()`, `scanf()` etc. These files must be included for working with these functions.
- Different function are declared in different header files.
 - For example C++ standard I/O functions are in 'iostream' header file whereas
 - Functions which perform string operations are in 'string' header file.

File Inclusion

Header File or Standard files:

- **Syntax:** `#include <file_name>`
 - where *file_name* is the name of file to be included.
 - The '`<`' and '`>`' brackets tells the compiler to look for the file in standard directory.

File Inclusion

User defined files:

- When a program becomes very large, it is good practice to divide it into smaller files and include it whenever needed. These types of files are user defined files.
- These files can be included as: `#include"filename"`

File Inclusion

#include<filename.h>

- By using this syntax, when we are including header file then it will be loaded from default directory i.e. C:\TC\INCLUDE.
- Generally, by using this syntax we are including pre-defined header files.

#include "filename.h"

- Generally, by using this syntax we are including user-defined header files.
- It is loaded from the current working directory.

Conditional Compilation Directives

- Are type of directives which helps
 - To compile a specific portion of the program or
 - To skip compilation of some specific part of the program
 - Based on some conditions.
- This can be done with the help of two preprocessing commands **'ifdef'** and **'endif'**.

Conditional Compilation Directives

Syntax:

```
#ifdef macro_name
```

```
statement1;
```

```
statement2;
```

```
statement3;
```

```
.
```

```
.
```

```
.
```

```
statementN;
```

```
#endif
```

- If the macro with name as '*macroname*' is defined
 - Then the block of statements will execute normally
 - Else the compiler will simply skip this block of statements.

Conditional Compilation Directives

```
#include <stdio.h>
#define YEARS_OLD 10
int main()
{
#ifdef YEARS_OLD
printf("TechOnTheNet is over %d years old.\n", YEARS_OLD);
#endif
printf("TechOnTheNet is a great resource.\n");
return 0;
}
```

Output

```
/tmp/PG7rWF95Ud.o
TechOnTheNet is over 10 years old.
TechOnTheNet is a great resource.
```

- In the C Programming Language, the `#ifdef` directive allows for conditional compilation.
- The preprocessor determines if the provided macro exists before including the subsequent code in the compilation process.

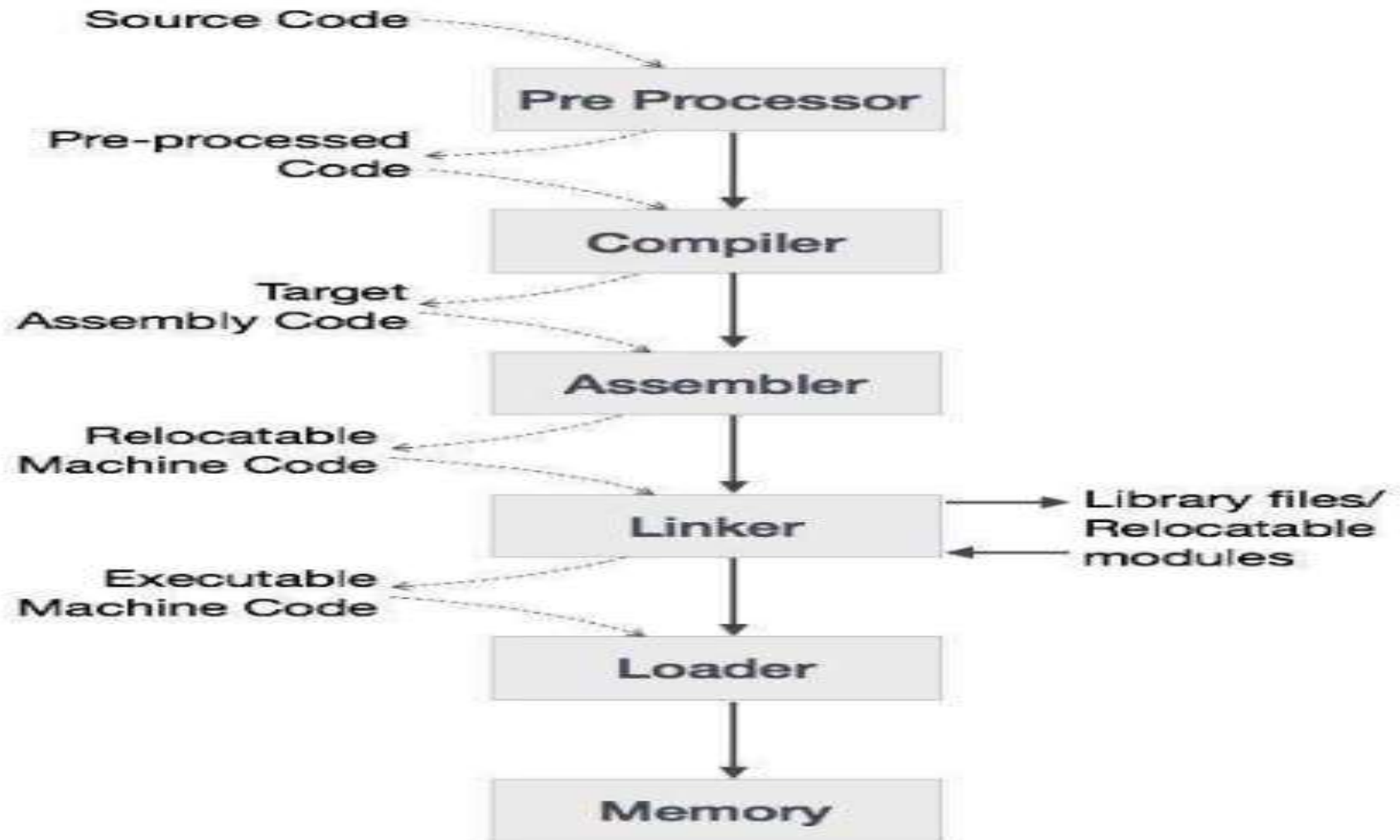
Other directives

- Apart from the above directives there are two more directives which are not commonly used.
- These are:
 - **#undef Directive**
 - **#pragma Directive**

Other directives

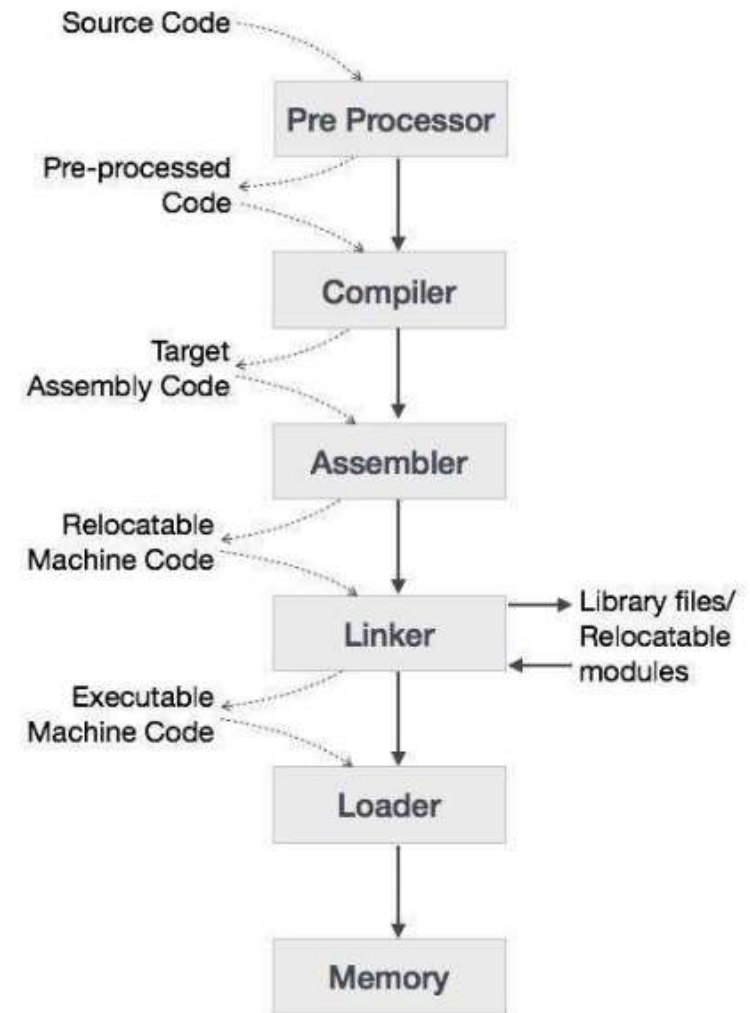
- **#undef Directive:**
 - The #undef directive is used to undefine an existing macro. This directive works as:
 - #undef LIMIT
 - Using this statement will undefine the existing macro LIMIT.
 - After this statement every “#ifdef LIMIT” statement will evaluate to false.
- **#pragma Directive:**
 - This directive is a special purpose directive and is used to turn on or off some features.
 - This type of directives are compiler-specific, i.e., they vary from compiler to compiler.

How a program, using C compiler, is executed on a host machine ?

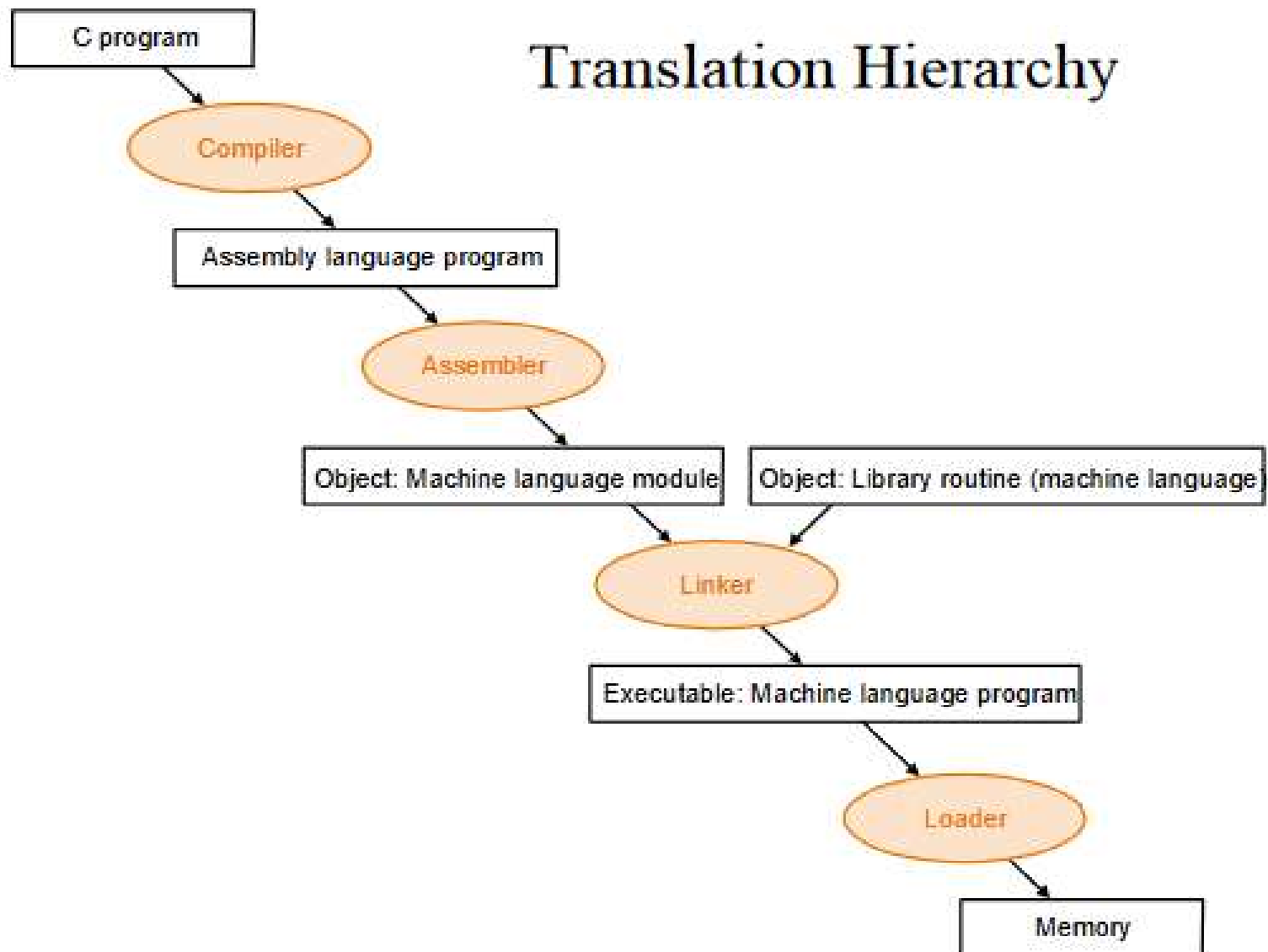


How a program, using C compiler, is executed on a host machine ?

- User writes a program in C language (high-level language).
- The C compiler, compiles the program and translates it to assembly program.
- An assembler then translates the assembly program into machine code (object file).
- A linker tool is used to link all the parts of the program together for execution (executable machine code).
- A loader loads all of them into memory and then the program is executed.

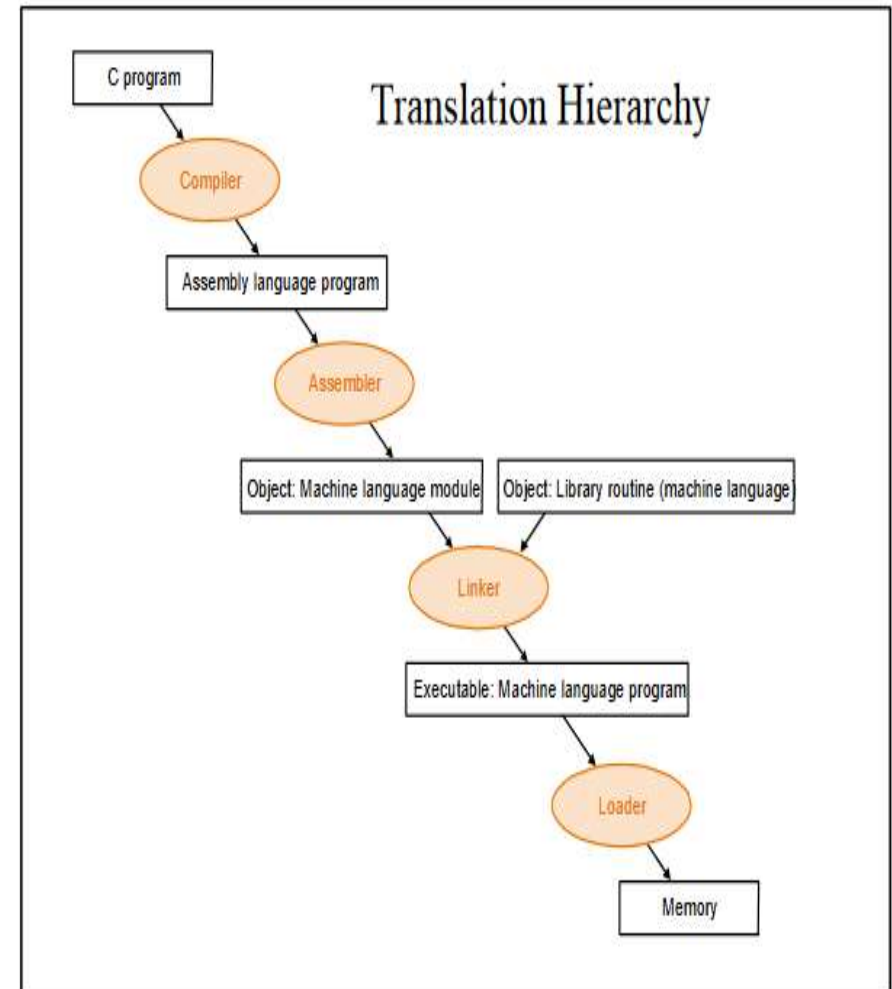


Translation Hierarchy



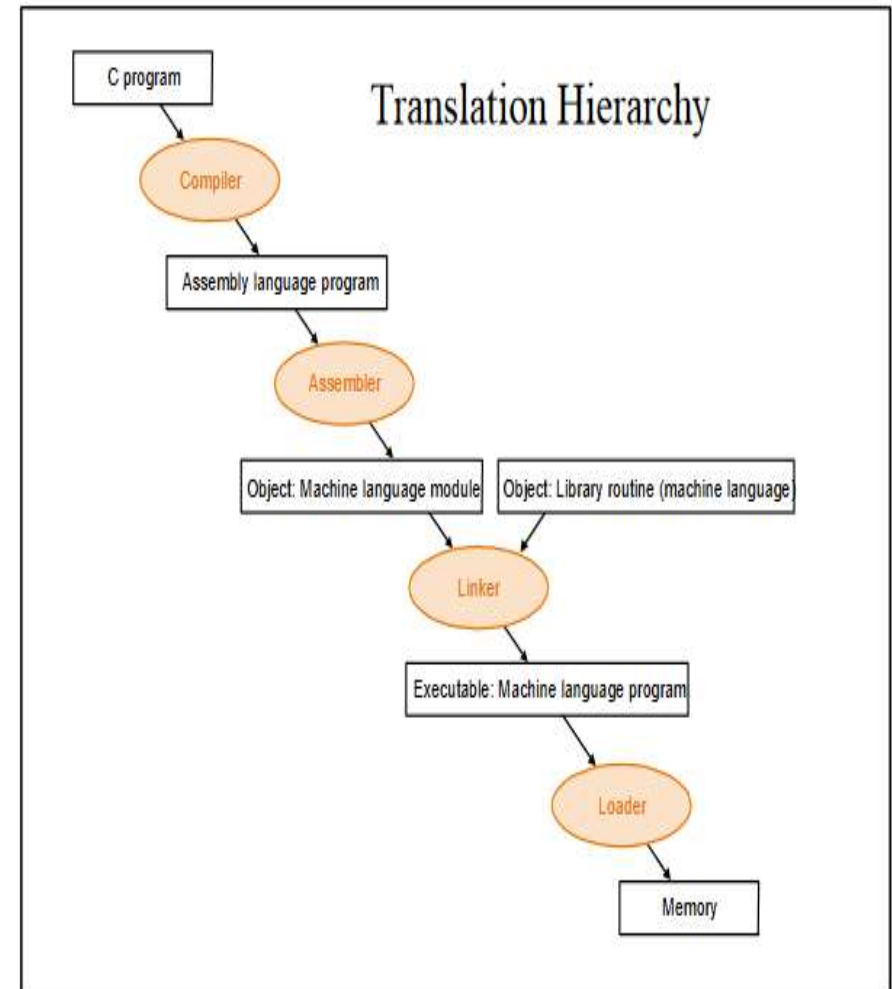
Compiler

- A compiler is a software that
 - translates the code written in one language to some other language
 - without changing the meaning of the program.



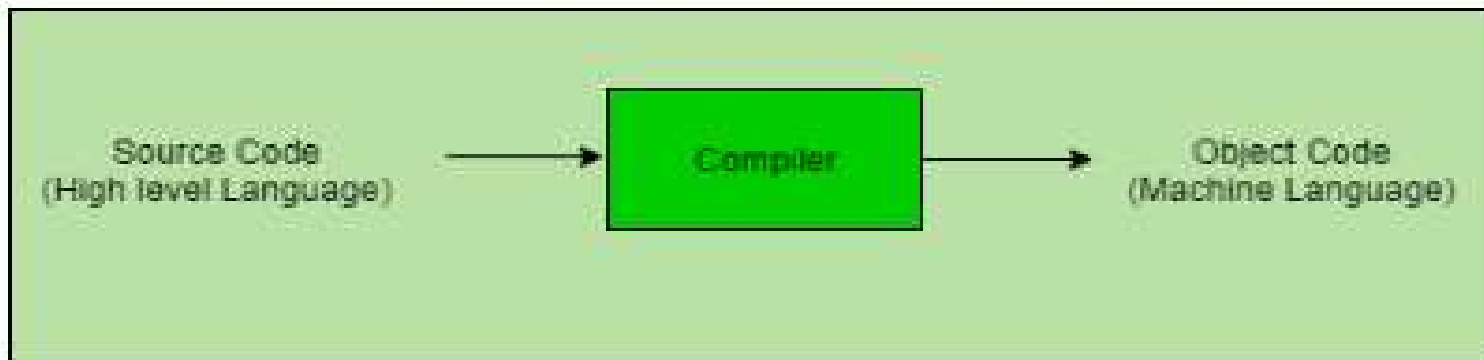
Compiler

- The compiler is also said to
 - make the target code efficient and
 - optimized in terms of time and space.
- In this example , “C” compiler converts high-level language program into assembly language



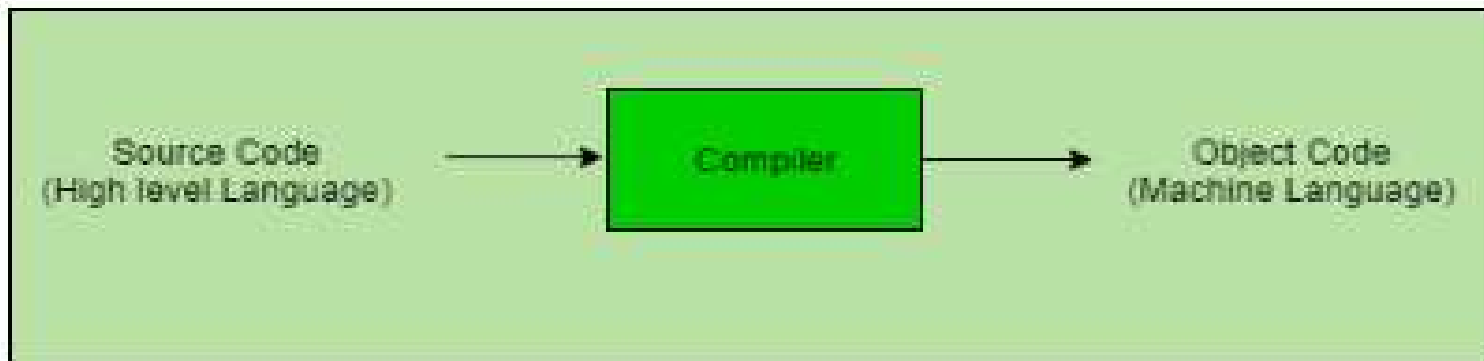
Compiler

- The language processor that reads the complete source program written in high level language **as a whole in one go** **and translates it** into an equivalent program in machine language is called as a Compiler.



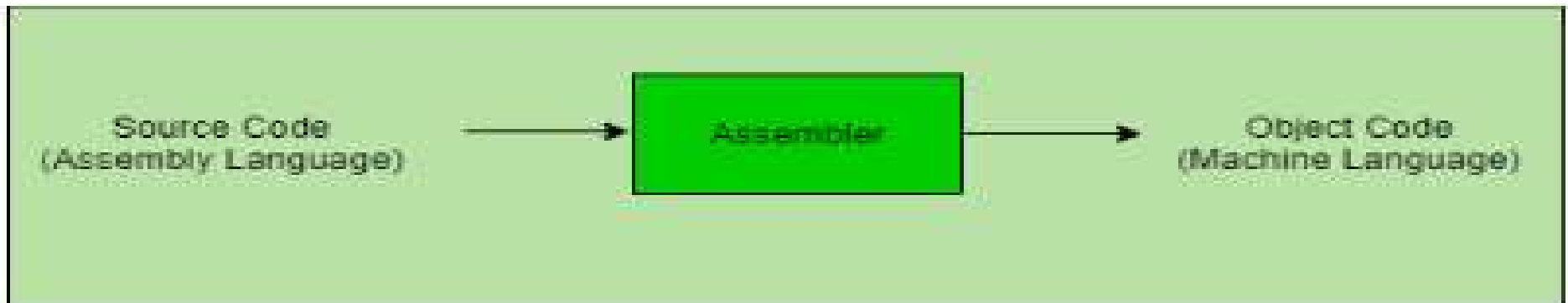
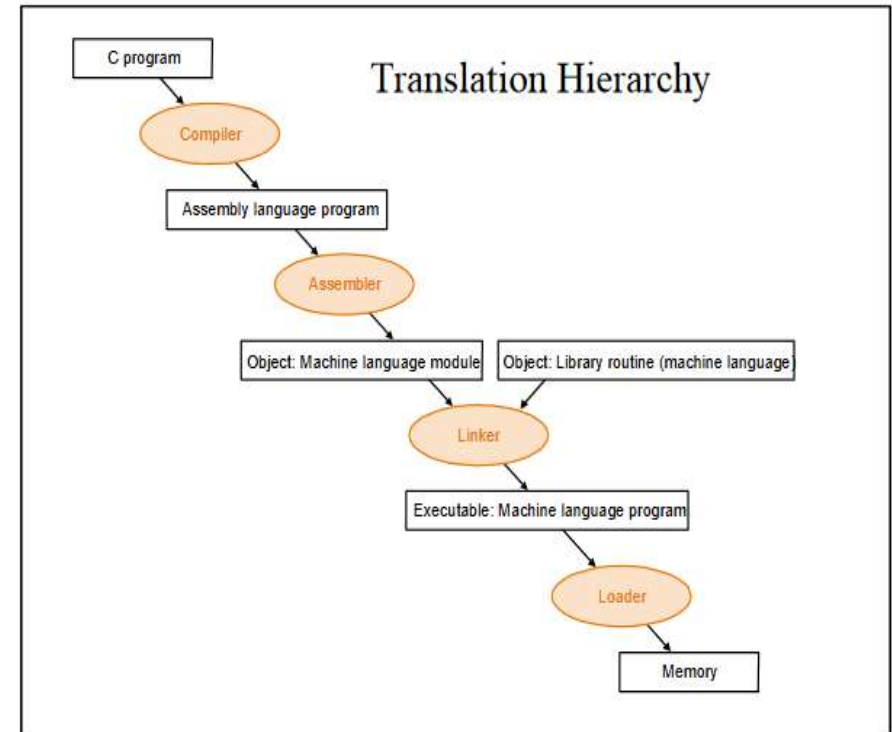
Compiler

- In a compiler, the source code is translated to object code successfully if it is free of errors.
- The compiler specifies
 - the errors at the end of compilation with line numbers when there are any errors in the source code.
 - The errors must be removed before the compiler can successfully recompile the source code again.



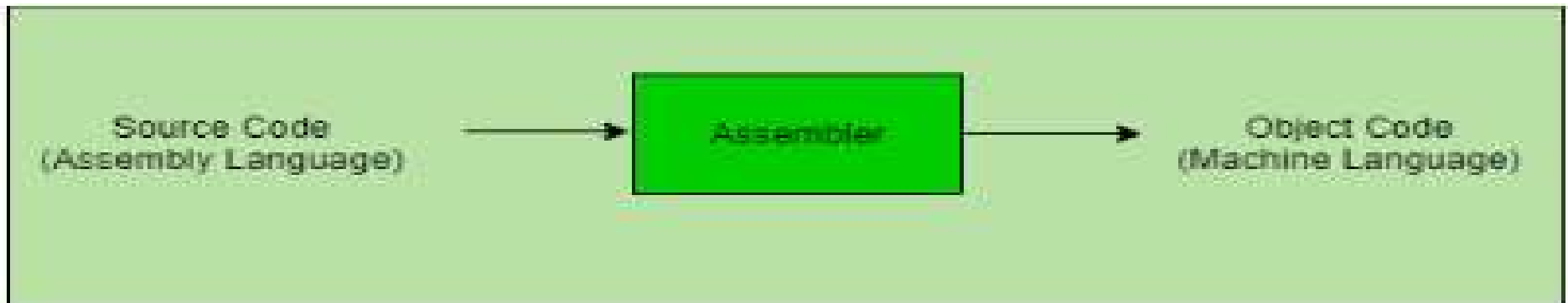
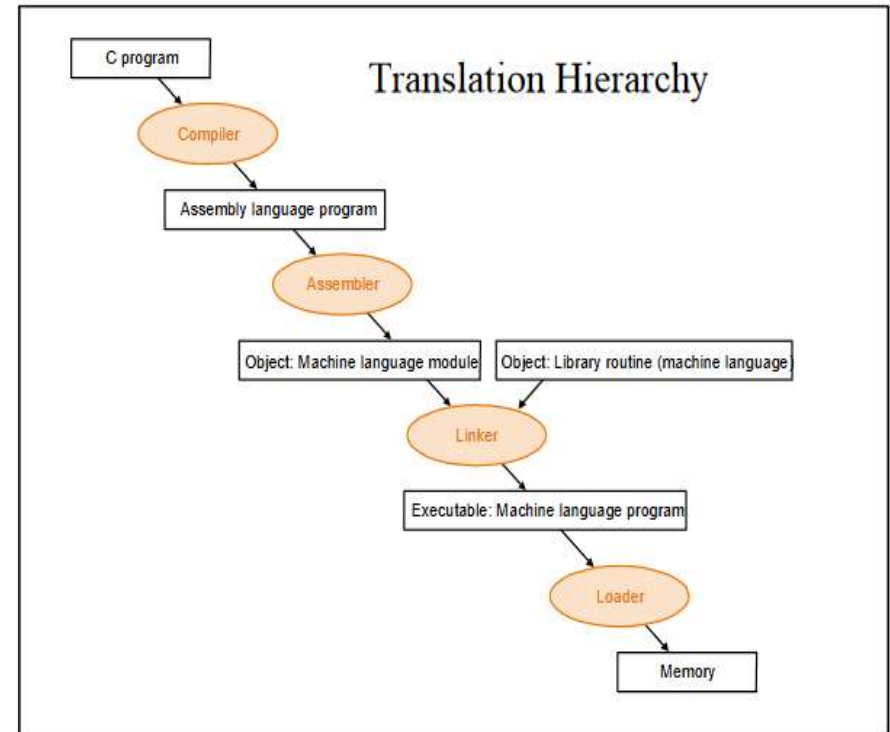
Assembler

- A program that converts assembly language programs into object files



Assembler

- The output of an assembler is called an object file
- Object files contain a combination of
 - machine instructions,
 - data, and
 - information needed to place instructions properly in memory.



Object file format

Object file header	Text segment	Data segment	Relocation information	Symbol table	Debugging information
--------------------	--------------	--------------	------------------------	--------------	-----------------------

- Object file header describes the size and position of the other pieces of the file
- Text segment contains the machine instructions
- Data segment contains binary representation of data in assembly file
- Relocation info identifies instructions and data that depend on absolute addresses
- Symbol table associates addresses with external labels and lists unresolved references
- Debugging info

Interpreter

- An interpreter, like a compiler, translates high-level language into low-level machine language.

Interpreter

- The difference lies in the way they read the source code or input.
 - A compiler reads the whole source code at once, creates tokens, checks semantics, generates intermediate code, executes the whole program and may involve many passes.
 - An interpreter reads a statement from the input, executes it, then takes the next statement in sequence.

Interpreter

- If an error occurs,
 - an interpreter stops execution and reports it. The interpreter moves on to the next line for execution only after removal of the error.
 - whereas a compiler reads the whole program even if it encounters several errors.
- **Example:** Perl, Python and Matlab.

COMPILER

A compiler is a program which converts the entire source code of a programming language into executable machine code for a CPU.

Compiler takes large amount of time to analyze the entire source code but the overall execution time of the program is comparatively faster.

Compiler generates the error message only after scanning the whole program, so debugging is comparatively hard as the error can be present anywhere in the program.

Generates intermediate object code.

Examples: C, C++, Java

INTERPRETER

interpreter takes a source program and runs it line by line, translating each line as it comes to it.

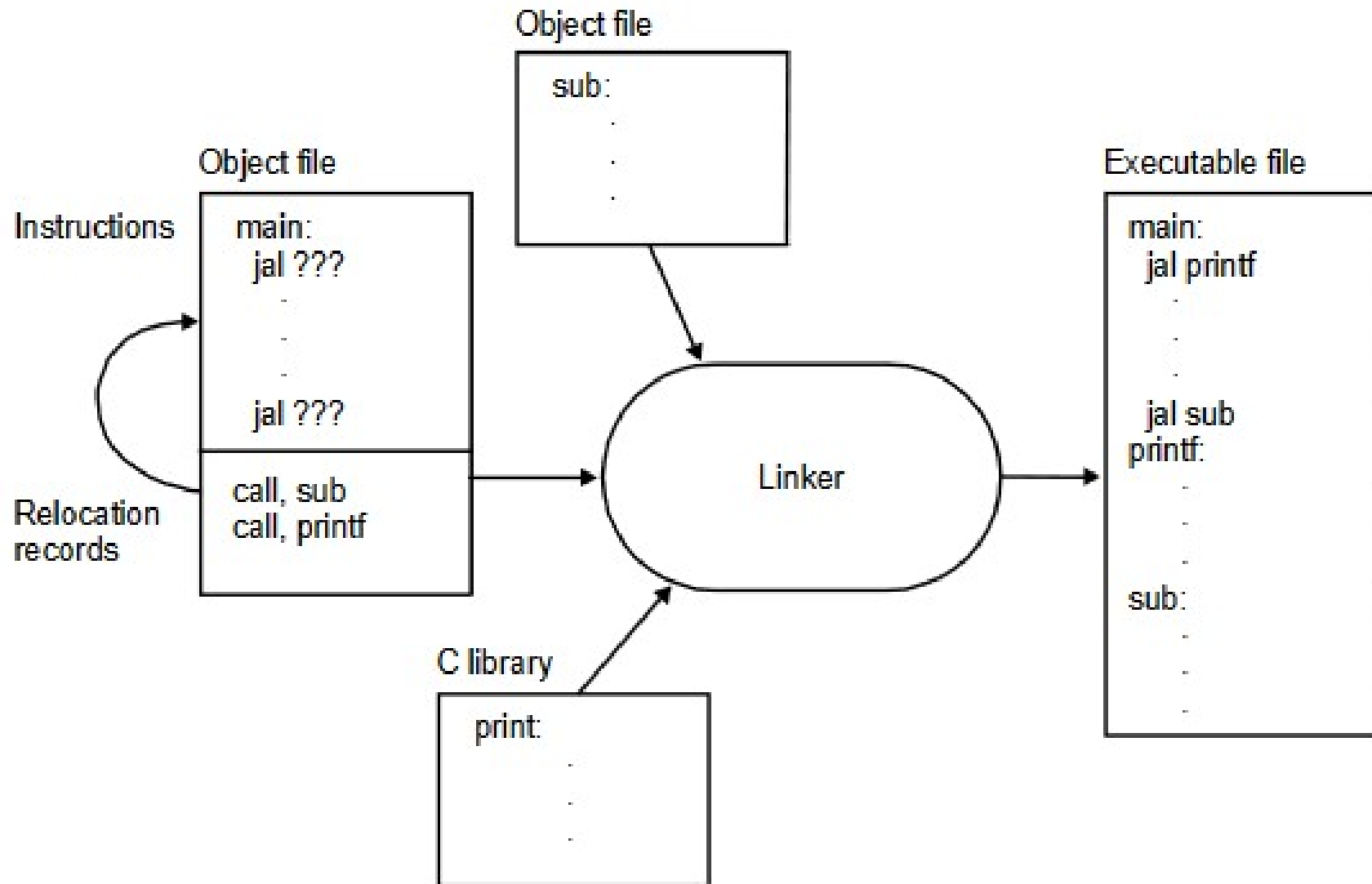
Interpreter takes less amount of time to analyze the source code but the overall execution time of the program is slower.

Its Debugging is easier as it continues translating the program until the error is met

No intermediate object code is generated.

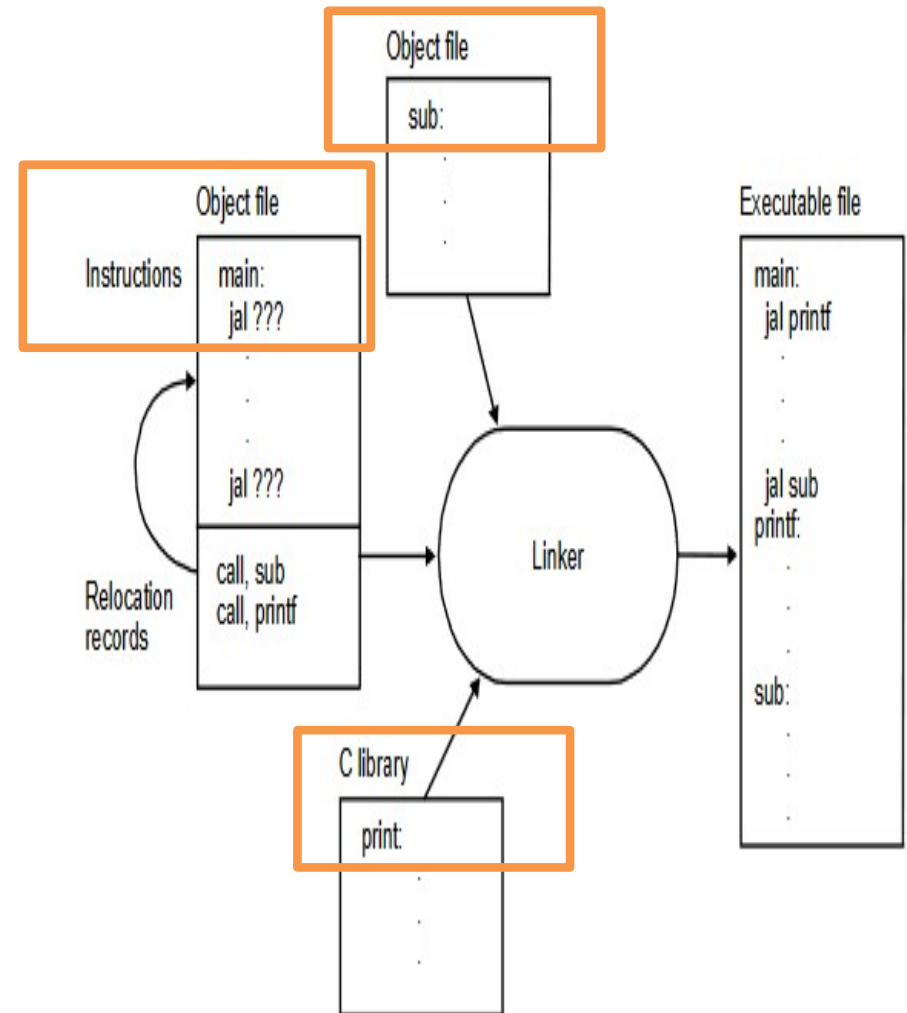
Examples: Python, Perl

Linker



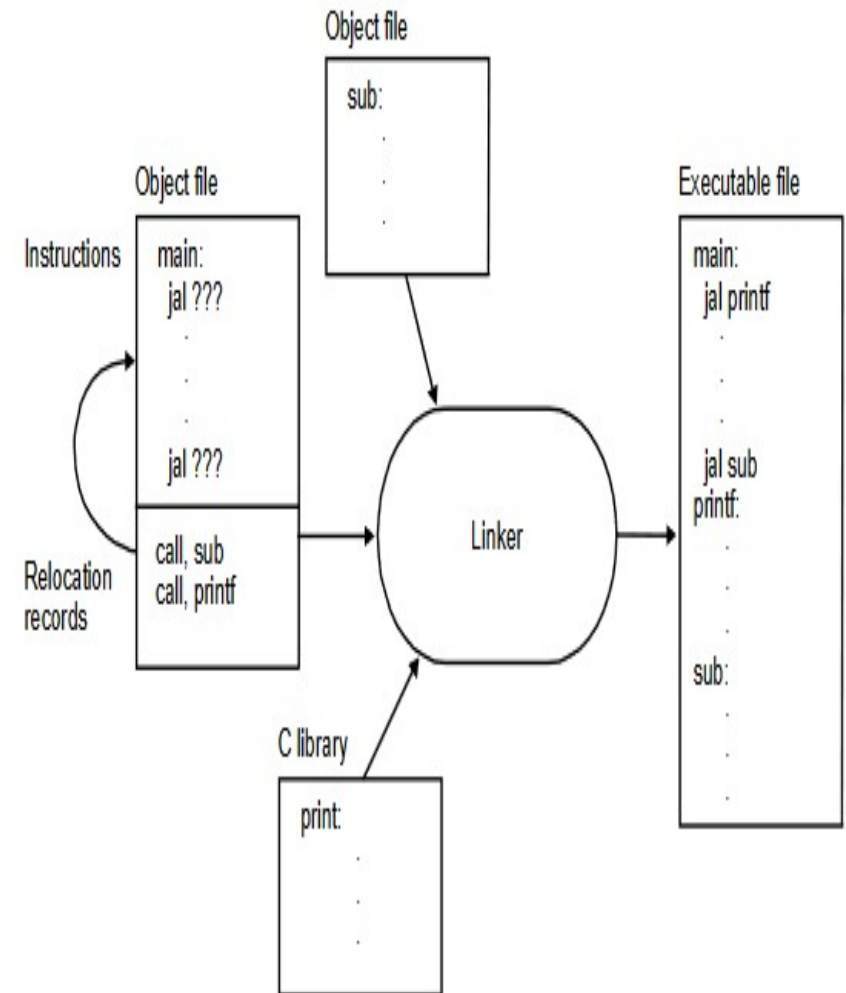
Linker

- Tool that merges the object files produced by separate compilation or assembly and creates a single executable file
- Linker are also called link editors.



Linker

- A linker tool is used to link all the parts of the program together for execution (executable machine code).



EXTERN STORAGE CLASS

PGM1.CPP

```
#include<stdio.h>
#include "pgm2.cpp"
int i;
void show(void);
int main()
{
    i=10;
    show();
    printf("\n Value of i in
pgm1.cpp=%d",i);
    return 0;
}
```

PGM2.CPP

```
extern int i;
void show()
{
    printf("\n value of i in
pgm2.cpp=%d",i);
}
```

EXTERN STORAGE CLASS

PGM1.CPP

```
#include<stdio.h>
#include "pgm2.cpp"
int i;
void show(void);
int main()
{
    i=10;
    show();
    printf("\n Value of i in
pgm1.cpp=%d",i);
    return 0;
}
```

PGM2.CPP

```
extern int i;
void show()
{
    printf("\n value of i in
pgm2.cpp=%d",i);
}
```

OUTPUT-

```
C:\TC\BIN>tc.exe
```

```
value of i in pgm2.cpp=10
Value of i in pgm1.cpp=10
```

Linker

- The major tasks–
 - To search and locate referenced module/routines in a program
 - Searches the program to find library routines used by program, e.g. printf(), math routines,...

Loader in C/C++

- **Loader** is the program of the **operating system**
 - which **loads the executable** from the disk into the **primary memory(RAM)** for execution.
- It allocates the memory space to the executable module in main memory and then transfers control to the beginning instruction of the program .

Loader in C/C++

- It places programs into memory and prepares them for execution.
- Loading a program involves reading the contents of the executable file containing the program instructions **into memory,**
- Then carrying out other required **preparatory tasks to prepare the executable for running.**
- Once loading is complete, the operating system starts the program by passing control to the loaded program code.

[https://en.wikipedia.org/wiki/Loader_\(computing\)](https://en.wikipedia.org/wiki/Loader_(computing))

Relocation

- The available memory is generally shared among a number of processes in a multiprogramming system,
- **So it is not possible to know in advance which other programs will be resident in main memory at the time of execution of this program.**
- Swapping the active processes in and out of the main memory enables the operating system to have a larger pool of ready-to-execute process.

-GeeksforGeeks

Relocation

- When a program gets swapped out to a disk memory, then it is not always possible that when it is swapped back into main memory then it occupies the previous memory location,
- Since the location may still be occupied by another process.
- We may need to relocate the process to a different area of memory. Thus there is a possibility that program may be moved in main memory due to swapping.

-GeeksforGeeks

<https://www.geeksforgeeks.org/requirements-of-memory-management-system/>

Relocation

- **Relocation** is the process of assigning load addresses for position-dependent code and data of a program and adjusting the code and data to reflect the assigned addresses.
- **Relocation is typically done by the linker at link time, but it can also be done at load time by a relocating loader.**

-Wikipedia

GATE CS 1998 | Question 25

In a resident- OS computer, which of the following system software must reside in the main memory under all situations?

- (A)** Assembler
- (B)** Linker
- (C)** Loader
- (D)** Compiler

GATE CS 1998 | Question 25

In a resident- OS computer, which of the following system software must reside in the main memory under all situations?

- (A) Assembler
- (B) Linker
- (C) Loader
- (D) Compiler

Answer: (C)

Explanation: Loader is the part of an operating system that is responsible for loading programs and libraries. it places the programs into memory and also prepares them for execution.

Loading a program involves tasks such as reading the contents of the executable file containing the program instructions into memory, and then carrying out other required preparatory tasks to prepare the executable for running so any software must reside in the main memory under all situations.

The operating system starts the program by passing control to the loaded program code once the loading process is completed.

Option (C) is correct.

Q) A linker program

- a. places the program in the memory for the purpose of execution.
- b. relocates the program to execute from the specific memory area allocated to it.
- c. links the program with other programs needed for its execution.
- d. interfaces the program with the entities generating its input data.

Q) A linker program

- a. places the program in the memory for the purpose of execution.
- b. relocates the program to execute from the specific memory area allocated to it.
- c. links the program with other programs needed for its execution.
- d. interfaces the program with the entities generating its input data.

Ans) c)

What is an Operating System?

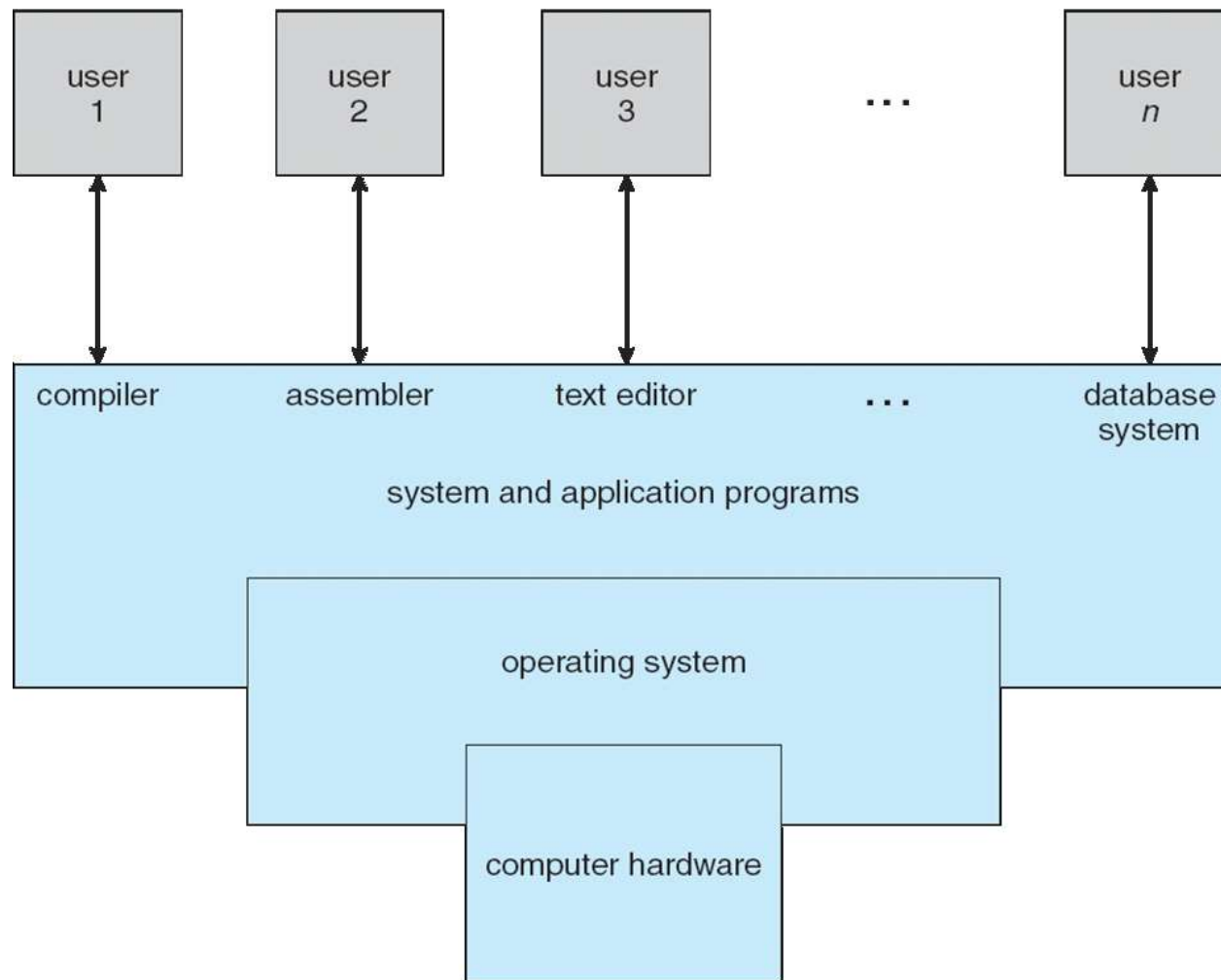
What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware.
- Provides an environment in which a user can execute programs in a convenient and efficient manner.

What is an Operating System?

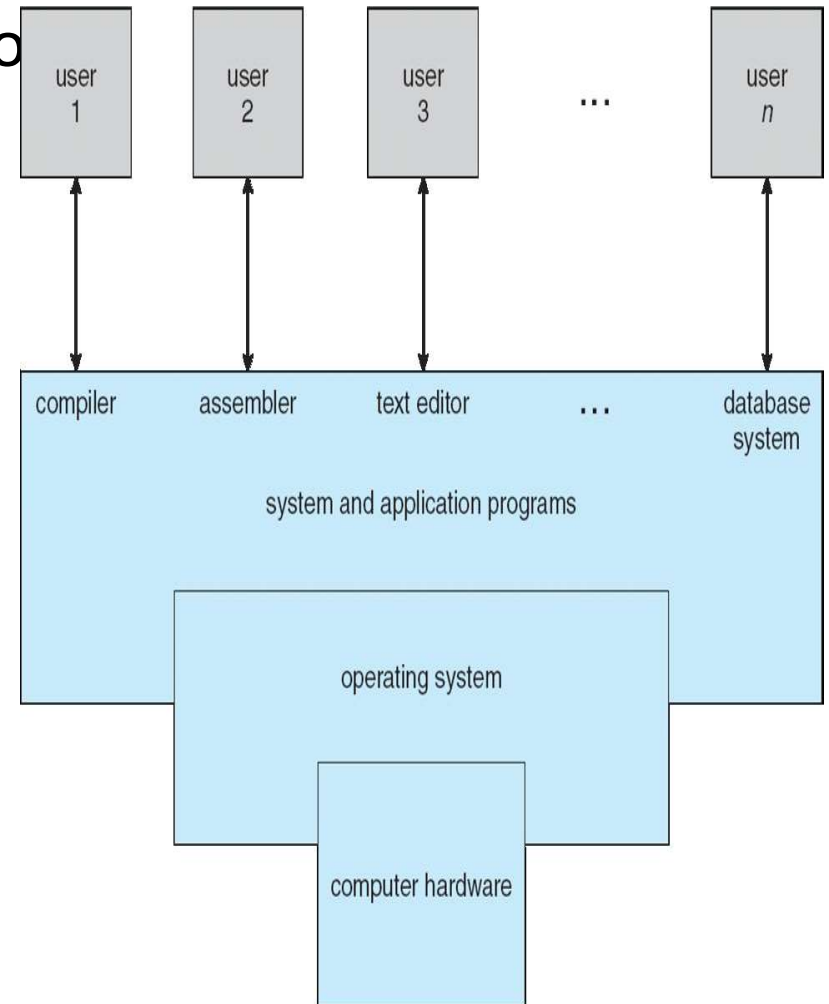
- Operating system goals:
 - Execute user programs and make solving user problems easier
 - Make the computer system convenient to use
 - Use the computer hardware in an efficient manner

Four Components of a Computer System



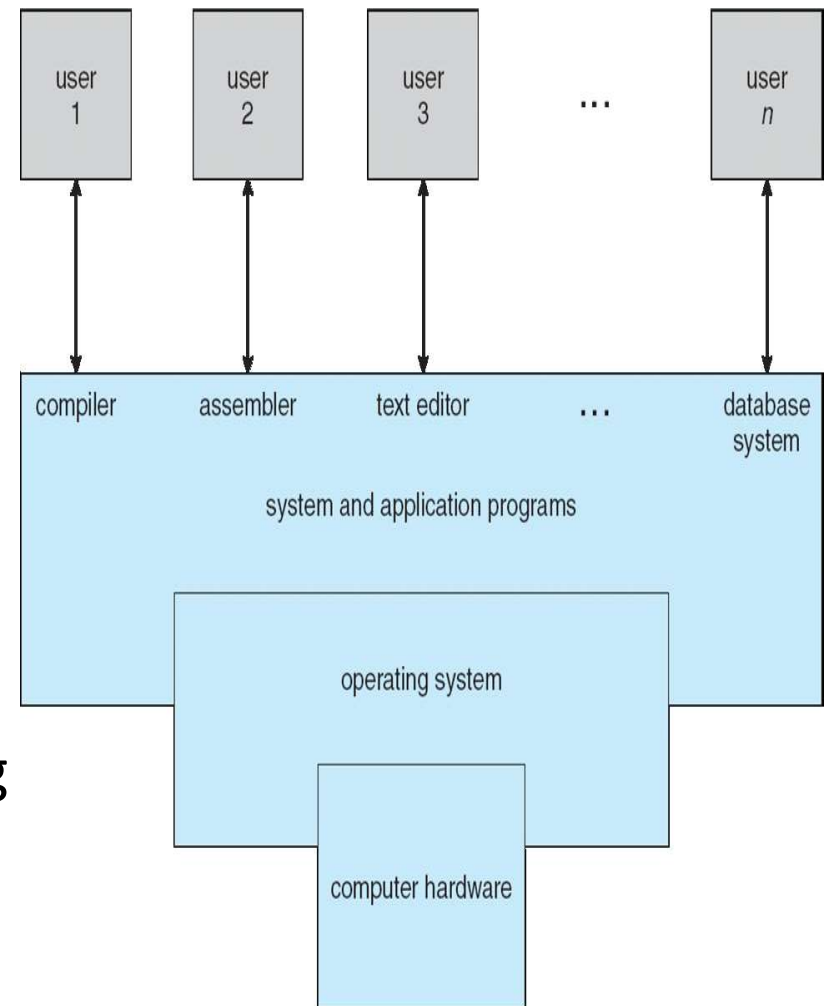
Computer System Structure

- Computer system can be divided into four components:
 - Hardware
 - Operating system
 - System & Application programs
 - Users



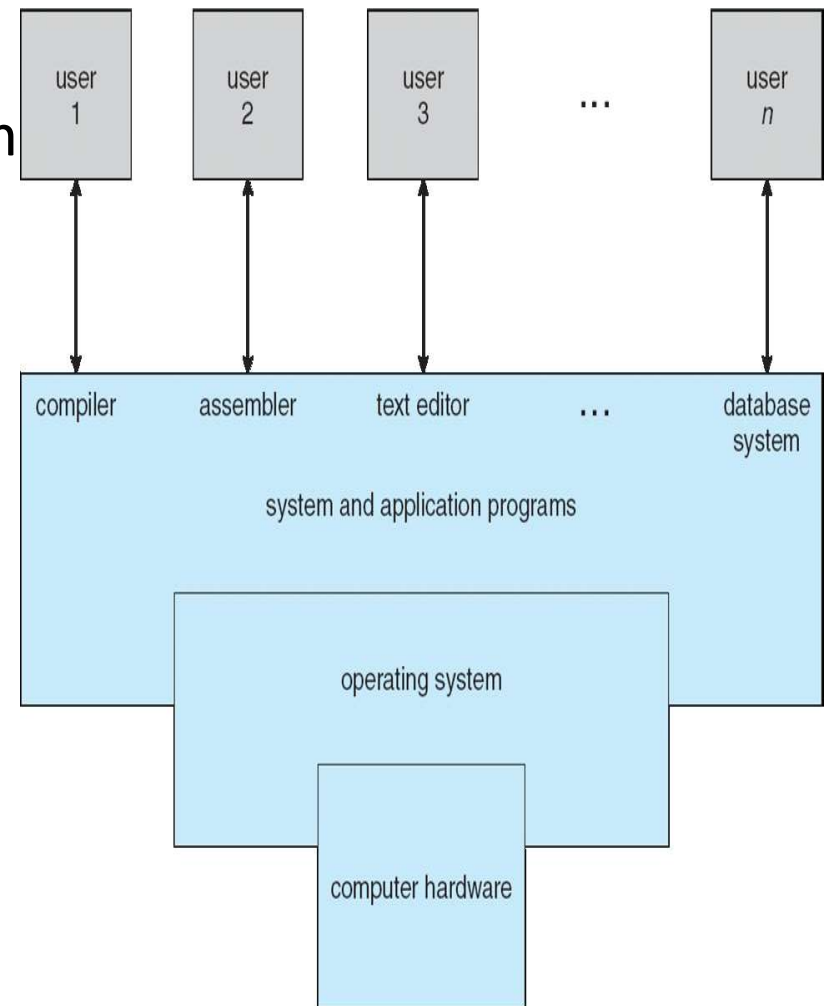
Computer System Structure

- Hardware – provides basic computing resources
 - CPU,
 - memory,
 - I/O devices
- Operating system
 - Controls and
 - Coordinates use of hardware among various applications and users

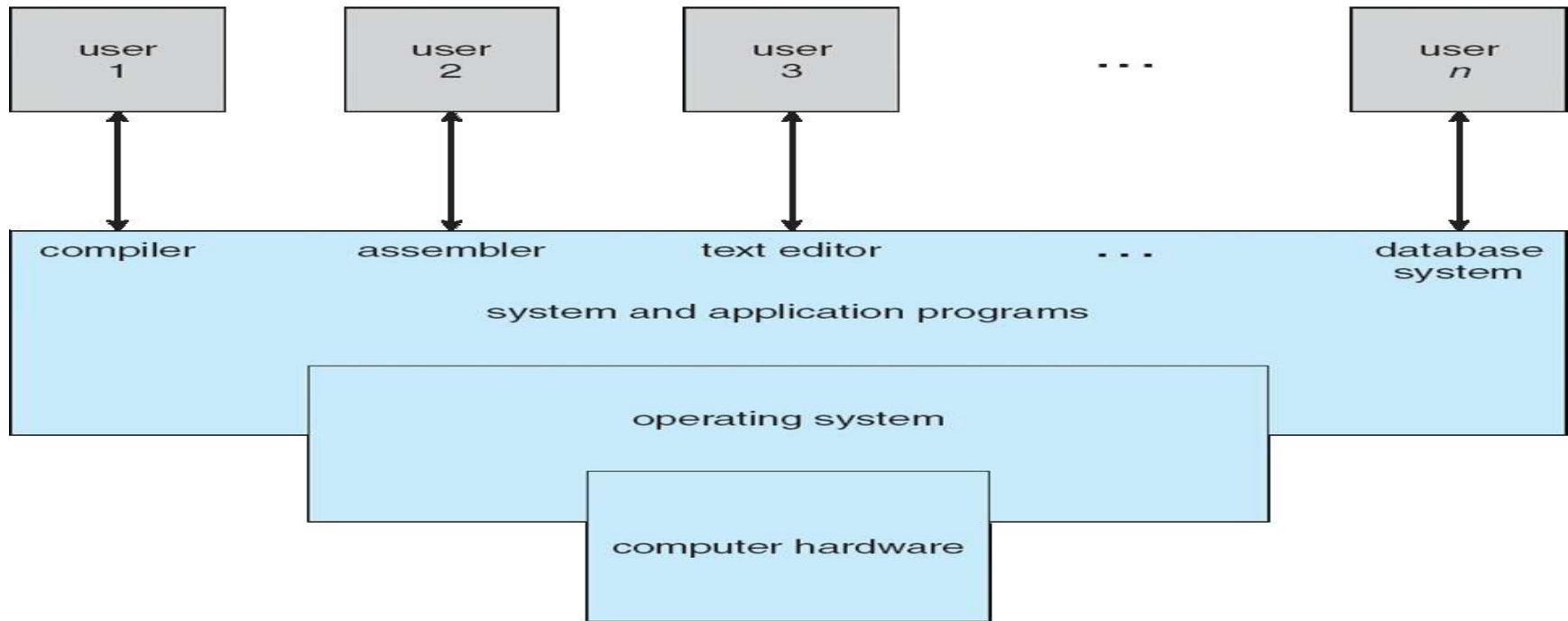


Computer System Structure

- Application programs
 - define the ways in which the system resources are used
 - to solve the computing problems of the users
 - Word processors,
 - web browsers,
 - database systems,
 - video games
- Users
 - People, machines, other computers



Four Components of a Computer System



OS controls and coordinates the use of hardware among the various application programs for the various users.